# Reposec: Security Review System

Andrea Scalora

## 1. Introduzione

Il presente documento descrive il progetto finale sviluppato per il percorso Academy "Cybersecurity & AI", incentrato sulla costruzione di un sistema **multi-agente** di Intelligenza Artificiale per la revisione automatizzata della sicurezza del codice sorgente. Il lavoro propone lo sviluppo di un **Security Review System** capace di analizzare repository applicative (ad esempio progetti GitHub) e produrre un report strutturato delle vulnerabilità individuate. Il sistema è composto da un team di **agenti specializzati** che collaborano per identificare rischi applicativi in ottica **OWASP e secure-by-design** e che forniscono raccomandazioni di mitigazione comprensibili a sviluppatori e stakeholder non tecnici.

## 2. Abstract

Questo progetto propone un sistema multi-agente basato sui Large Language Models per la code security review automatizzata di repository software, con l'obiettivo di supportare sviluppatori e team di sicurezza nell'identificazione precoce di vulnerabilità applicative.

L'architettura si fonda su più agenti LLM specializzati (analisi backend, frontend, configurazioni, dipendenze, compliance) orchestrati in pipeline, che esaminano il codice, i log e i file di progetto producendo finding classificati secondo le categorie OWASP Top 10 e corredati da suggerimenti di remediation.

Il sistema, implementato utilizzando framework per agenti e strumenti di code-assistant, introduce un **ciclo di apprendimento**: Tramite Cognee, ogni scansione arricchisce la memoria permanente con nuovi esempi, migliorando le analisi successive. Ad ogni scansione effettuata, per ogni remediation proposta che viene approvata da SonarQube il sistema introduce in memoria un nuovo problema di sicurezza mappandolo con quelli OWASP esistenti.

# 3. Contesto

Il panorama della sicurezza informatica contemporanea è caratterizzato da una crescente complessità delle applicazioni web e dalla proliferazione di minacce mirate al codice sorgente. Le vulnerabilità applicative, classificabili secondo standard di riferimento come l'OWASP Top 10, continuano a rappresentare uno dei vettori di attacco più sfruttati. Gli strumenti di analisi statica (SAST) tradizionali, pur offrendo un supporto valido, presentano limitazioni significative: elevati tassi di falsi positivi, incapacità di comprendere il contesto semantico del codice, e difficoltà nell'adattarsi a pattern architetturali moderni.
L'intelligenza artificiale, e in particolare i Large Language Models (LLM), emerge come strumento innovativo per superare questi limiti. Grazie alla capacità di comprendere il contesto, ragionare su snippet di codice e fornire spiegazioni in linguaggio naturale, i modelli linguistici possono supportare gli sviluppatori nell'individuazione precoce di difetti di sicurezza. Il paradigma dei sistemi multi-agente, in cui diversi specialisti collaborano per completare un task complesso, si presta particolarmente bene alla security review, permettendo un'analisi approfondita e modulare del codice sorgente.

# 4. Idea Progettuale

Questo Multi-Agent Security Reviewer analizza repository GitHub in modalità **read-only** e produce un report strutturato delle vulnerabilità di sicurezza individuate.

## 4.1 Architettura - Fase 1

| Componente | Tecnologia | Note |
|---|---|---|
| LLM Orchestrator | OpenCode SDK | Sessioni isolate per agente |
| Documentazione | Web search nativa | Per consultare docs aggiornate |
| Memory | File Markdown | OWASP TOP 10 e principi SBD |

Procedere poi ad implementare le varie feature una ad una: GitHub MCP, SonarQube MCP e Cognee con relative dipendenze.

## 4.2 Architettura - Fase Finale

| Componente | Tecnologia | Ruolo |
|---|---|---|
| LLM Orchestrator | OpenCode SDK (Python) | Sessioni isolate per agente |
| Documentazione | Context7 MCP | Docs librerie aggiornate |
| Validazione | SonarQube MCP | Analisi statica indipendente |
| GitHub Integration | GitHub MCP | Issue, PR, commit log |
| Memory/Knowledge | Cognee MCP + LanceDB + Kuzu | Memoria permanente findings |
| Riferimenti | OWASP Top 10 2025 + SbD Framework | Standard di sicurezza |
| Output | Markdown + Issue GitHub | Report strutturato |

## 4.3 Flusso Completo

Il sistema è composto da agenti specializzati che collaborano in pipeline:

[Input: Repo GitHub URL]
→ [**Ingest Agent**: Clona repo, genera fingerprint tecnologico (stack tecnologico, framework, struttura)]
→ [Agenti paralleli: Backend, Frontend, Secrets, Deps, Compliance]
├── [Context7 MCP] (docs librerie)
├── [GitHub MCP] (commit history)
├── [Web Search] (riferimenti)
└── [Knowledge Base] (riferimenti OWASP e secure-by-design)
→ [SonarQube MCP valida]
→ [Cognee salva in Knowledge Graph]
→ [Aggregator consolida]
→ [Report Markdown + Issue GitHub]

## 4.4 Agenti Specializzati

| Agente | Responsabilita | Focus |
|---|---|---|
| Ingest Agent | Clona repo, identifica stack | Fingerprint tecnologico |
| Backend Security | API, auth, database, logging | A01-A10 backend |
| Frontend Security | XSS, CORS, cookie, sanitizzazione | A01-A10 frontend |
| Secrets & Config | Hardcoded secrets, permessi, config | A01, A04, A05 |
| Dependency Risk | Librerie, CVE, versioni vulnerabili | A06 |
| Compliance | Mappa su OWASP + SbD | Compliance mapping |
| Aggregator | Consolida, deduplica, genera report | Output finale |

L'output è un **security_report.md** contenente: executive summary, findings categorizzati per severità, riferimenti OWASP, e suggerimenti di remediation.

## 4.5 Ciclo di Apprendimento

[Scansione N] → [Trova vulnerabilità] → [Cognee salva: categoria OWASP + codice]
[Scansione N+1] → [Cognee recupera esempi simili] → [Analisi arricchita da memoria storica]

# 5. Risorse OWASP

## 5.1 OWASP Top 10 2025

Classificazione delle vulnerabilità applicative più critiche

| | |
|---|---|
| A01:2025 | Broken Access Control |
| A02:2025 | Cryptographic Failures |
| A03:2025 | Injection |
| A04:2025 | Insecure Design |
| A05:2025 | Security Misconfiguration |

| | |
|---|---|
| A06:2025 | Vulnerable and Outdated Components |
| A07:2025 | Identification and Authentication Failures |
| A08:2025 | Software and Data Integrity Failures |
| A09:2025 | Security Logging and Monitoring Failures |
| A10:2025 | Server-Side Request Forgery |

## 5.2 OWASP Secure-by-Design Framework

I principi fondamentali di progettazione sicura.

| | |
|---|---|
| 1 | Minimize attack surface |
| 2 | Least privilege |
| 3 | Secure defaults |
| 4 | Defense in depth |
| 5 | Fail securely |
| 6 | Zero trust |
| 7 | Separation of duties |
| 8 | Avoid security by obscurity |
| 9 | Simplicity |
| 10 | Root cause correction |

# 6. Casi d'Uso

| Caso d'Uso | Descrizione |
|---|---|
| **Security Review Pre-Deploy** | Uno sviluppatore o team DevOps desidera analizzare un repository prima del rilascio in produzione. Il sistema esegue una scansione completa del codice e produce un report dettagliato delle vulnerabilità, permettendo di identificare e correggere i difetti critici prima del deploy. |
| **Analisi di Repository Legacy** | Un'azienda ha un codebase storico mai analizzato per la sicurezza. Il sistema esegue una scansione per individuare vulnerabilità note, secrets esposti e dipendenze problematiche, fornendo una baseline di sicurezza prioritaria. |
| **Monitoraggio Continuo della Sicurezza** | Integrazione del sistema in una CI/CD pipeline per analisi automatica ad ogni commit o pull request, con report di sicurezza integrato nel flusso di sviluppo. |
| **Compliance Reporting per Audit** | Il sistema produce report strutturati che mappano i finding su standard OWASP, facilitando la preparazione di audit di sicurezza e la documentazione per requisiti normativi. |

# 7. MCP Servers

## 7.1 Context7 MCP

- Docs librerie/framework aggiornate
- Rate limit: 500 richieste/mese (gratis)
- Alternativa: web search nativa

## 7.2 SonarQube MCP

- Quality gate status
- Security issues e hotspots
- Validazione indipendente dei finding

## 7.3 GitHub MCP

- Lettura repository
- Creazione issue automatizzate
- Accesso a commit history, PR

## 7.4 Cognee MCP (Sistema Ideale)

- Memoria permanente: tutti i findings
- Knowledge graph: categorie OWASP + principi SbD
- Retrieval semantico: trova esempi simili

# 8. Osservazioni Preliminari

La scelta di un'**architettura multi-agente** rispetto a un singolo LLM generico risponde a due esigenze fondamentali: specializzazione e aumento della precisione. Agenti dedicati (backend, frontend, config, dipendenze, compliance) permettono **analisi mirate** con prompt ottimizzati per ciascun dominio, migliorando la qualità dei finding rispetto a un approccio "one-size-fits-all".

## 8.1 Limitazioni e Guardrails

È fondamentale riconoscere i limiti intrinseci di un sistema basato su LLM:

- **Falsi Positivi**: il modello può segnalare vulnerabilità inesistenti a causa di incomprensione del contesto.
- **Assunzioni implicite**: il modello può fare assunzioni errate su librerie, framework o pattern non familiari.
- **Dipendenza LLM**: costi API e latenza.
- **Context7**: rate limitati (versione cloud).

Per ridurre i rischi sopra citati, il sistema implementa:

- **Modalità Read-Only**: nessuna modifica al repository analizzato.
- **Guardrails**: divieto di eseguire comandi pericolosi (rm, curl, exec) sul sistema ospite.
- **Output Strutturato**: formato Markdown standardizzato per il report.
- **Aggregator con Deduplica**: consolidamento dei finding per ridurre ridondanze.

Il sistema è concepito come strumento di **supporto** allo sviluppatore, non come sostituto di un security expert. I risultati devono essere **validati** da personale qualificato prima di intraprendere azioni correttive.

# 9. Studio dello Stato dell'Arte

## 9.1 Ricerca Accademica Recente

Negli ultimi anni, la comunità scientifica ha esplorato l'applicazione degli LLMs alla security review automatizzata. Ricerche come **AutoReview, AutoSafeCoder, CodeAgent** e **RepoAudit** hanno dimostrato la fattibilità di utilizzare LLM per identificare vulnerabilità nel codice sorgente, sebbene con limitazioni in termini di copertura e accuratezza:

- **AutoReview**: Chen et al. (2025) presentano un sistema multi-agente LLM per security code review orientato a specifiche categorie di vulnerabilità. Il sistema implementa agenti specializzati per diversi aspetti della review di sicurezza.
- **AutoSafeCoder**: Nunez et al. (2024) propongono un framework multi-agent da University of Texas con tre agenti (Coding Agent, Static Analyzer Agent, Fuzzing Agent) che collaborano per generazione codice sicuro attraverso analisi statica e fuzz testing dinamico.
- **CodeAgent**: Tang et al. (2024) introducono un sistema multi-agente LLM per code review automation che emula la natura collaborativa della revisione umana. Il sistema utilizza agenti comunicanti specializzati per diversi aspetti della review (quality, security, style) con meccanismi di discussione iterativa.

## 9.2 Soluzioni Enterprise

Sul piano delle soluzioni enterprise, strumenti come **GitHub Code Scanning**, **Snyk Code** e **SonarQube** integrano motori AI per la detection di vulnerabilità. Questi strumenti si concentrano principalmente su analisi statica tradizionale arricchita con machine learning, mentre il nostro approccio esplora il paradigma multi-agente con interfaccia conversazionale:

- **GitHub Code Scanning**: Integra CodeQL con AI per detection avanzata, limitato a pattern noti e con limitata capacità di spiegazione contestuale.
- **SonarQube**: Piattaforma SAST con 5,000+ rules e integrazione AI. Offre MCP server per integrazione con agenti (adottato nel nostro progetto per validazione indipendente).
- **Snyk Code**: Scanner SAST basato su semantic analysis con focus su dependency vulnerabilities.

## 9.3 Analisi Approfondita: RepoAudit

**RepoAudit** rappresenta lo stato dell'arte per agenti LLM autonomi nel code auditing a livello di repository.
Architettura:

- **Memory Module**: Memorizza data-flow facts e contesto di esecuzione.
- **Exploration Engine**: Navigazione on-demand del codebase analizzando percorsi di esecuzione fattibili.
- **Validator Module**: Mitigazione delle allucinazioni attraverso verifica dei data-flow facts e checking della soddisfacibilità delle path conditions.

**Risultati Sperimentali**:

- **Precisione**: *78.43%* su 15 progetti benchmark real-world
- **Costo medio**: $2.54 e 0.44 ore per progetto (OpenAI GPT-4o)
- **Nuovi bug trovati**: *185* in progetti high-profile (*174 confermati* o fixed)

Si differisce dal nostro approccio in questi punti:

| Caratteristica | RepoAudit | Reposec |
|---|---|---|
| **Focus** | Bug detection generico | Security-specific review |
| **Agenti** | Singolo agente con memoria | Multi-agente specializzati |
| **Validazione** | Validator module interno | SonarQube MCP + Cross-checking multi-agente |
| **Contesto** | Data-flow analysis | Context7 MCP (documentazione librerie aggiornata) |
| **Framework** | Custom agent framework | OpenCode SDK (orchestrazione strutturata) |
| **Integrazione** | Standalone | GitHub MCP (auto-creation issues) |
| **Memoria** | Session-based | Cognee MCP (knowledge graph persistente) |
| **Apprendimento** | Limitato | Comparazione cross-scan con retrieval storico |

## 9.4 Limitazioni LLM nella Security Review

Uno studio empirico "Security Code Review by LLMs: A Deep Dive into Responses" (Yu et al. 2024) su 6 LLM mostra che:

- GPT-4 **supera** significativamente i tool di analisi statica tradizionali.
- Ma genera risposte **verbose** e **non compliant** nel 20% dei casi.
- Performance migliori con **CWE list** come riferimento.
- Difficoltà con file di grandi dimensioni (>**token limit**).

La ricerca “LLM Hallucinations in Code Review” (diffray.ai 2025) mostra che:

- Il 29-45% del codice AI-generated contiene vulnerabilità di sicurezza.
- 19.7% delle raccomandazioni di package sono inesistenti (hallucinations).
- Mitigazioni combinate possono ridurre allucinazioni fino al 96%.

## 9.5 Distinzione di Reposec

Rispetto allo stato dell’arte, Reposec si distingue per:

1. **Architettura Multi-Agente Esplicita**: A differenza di CodeAgent (agenti comunicanti generici) e RepoAudit (singolo agente), il nostro sistema definisce agenti specializzati per domini di sicurezza specifici con orchestrazione strutturata.
2. **Integrazione Secure-by-Design**: Mappatura esplicita su OWASP Top 10 2025 e Secure-by-Design Framework, non presente negli altri sistemi.
3. **Ecosistema MCP**: Utilizzo del Model Context Protocol per integrazione standardizzata con tool esterni (SonarQube, GitHub, Context7, Cognee), garantendo interoperabilità e manutenibilità.
4. **Apprendimento Incrementale**: Confronto automatico tra scansioni successive attraverso Cognee knowledge graph, con tracking di vulnerabilità ricorrenti e regression.
5. **Output Multi-Stakeholder**: Report strutturati comprensibili sia per sviluppatori che per stakeholder non-tecnici, con auto-creation di issue GitHub.

## 9.6 Gap Addressati

1. **Falsi Positivi**: Attraverso validazione SonarQube e cross-checking multi-agente.
2. **Hallucinations**: Mitigate via validatore esterno e memoria persistente con Cognee.
3. **Context Limitation**: Context7 MCP per documentazione librerie aggiornata.
4. **Integration**: GitHub MCP per workflow CI/CD nativo.
5. **Scalabilità**: Architettura modulare che può processare codebase di grandi dimensioni.
6. **Explainability**: Output strutturati con riferimenti OWASP e spiegazioni contestuali.

# 10. Riferimenti

- OWASP Top 10 2025: https://owasp.org/Top10/2025/.
- OWASP Secure-by-Design: https://owasp.org/www-project-secure-by-design-framework/.
- OpenCode SDK: https://github.com/anomalyco/opencode-sdk-python.
- Cognee: https://github.com/topoteretes/cognee.
- SonarQube MCP: https://github.com/SonarSource/sonarqube-mcp-server.
- GitHub MCP: https://github.com/github/mcp-server.

**Riferimenti Bibliografici**

1. Chen, Y. et al. (2025). "AutoReview: An LLM-based Multi-Agent System for Security Issue-Oriented Code Review". In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE 2025). DOI: 10.1145/3696630.3728618.

2. Nunez, A., Islam, N.T., Jha, S., & Rad, P. (2024). "AutoSafeCoder: A Multi-Agent Framework for Securing LLM Code Generation through Static Analysis and Fuzz Testing". arXiv preprint arXiv:2409.10737.
3. Tang, X., Kim, K., Song, Y., Lothritz, C., Li, B., Ezzini, S., Tian, H., Klein, J., & Bissyande, T.F. (2024). "CodeAgent: Autonomous Communicative Agents for Code Review". arXiv preprint arXiv:2402.02172v5.
4. Sharanarthi, T. & Polineni, S. (2025). "Multi-Agent LLM Collaboration for Adaptive Code Review, Debugging, and Security Analysis". In: 2025 International Conference on Mechatronics, Robotics, and Artificial Intelligence (MRAI). DOI: 10.1109/MRAI65197.2025.11135756.
5. Yu, J., Liang, P., Fu, Y., Tahir, A., Shahin, M., Wang, C., & Cai, Y. (2024). "Security Code Review by LLMs: A Deep Dive into Responses". arXiv preprint arXiv:2405.12106. OpenReview.
6. diffray.ai (2025). "LLM Hallucinations Pose Serious Risks for AI Code Review". Research Deep-Dive, December 2025.
7. SonarSource. "SonarQube MCP Server". GitHub Repository: https://github.com/SonarSource/sonarqube-mcp-server
8. Guo, J., Wang, C., Xu, X., Su, Z., & Zhang, X. (2025). "RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing". In: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025). OpenReview. Available at: https://github.com/PurCL/RepoAudit.

---

**Nota**: Tutti i paper accademici sono stati presentati a conferenze top-tier (FSE, ICML) o pubblicati come preprint su arXiv/OpenReview nel 2024-2025, garantendo attualità e rilevanza scientifica.